PROJECT SPECIFICATION

AETHER LITE

Autonomous Multi-Agent Urban Simulation & Decision Intelligence Platform

“Simulate. Understand. Predict. Transform.”

PROJECT CREATORS	Louay Hamdi & Yassine Mezghani
TECHNICAL DOMAIN	Java · Artificial Intelligence · Simulation · Data Processing · Visualization
SPECIFICATION VERSION	Version 1.0 — Academic Project Specification (2026)
CORE ARCHITECTURE	Java Simulation Core · Spring Boot · WebSockets · PostgreSQL

LIGHTWEIGHT SCOPE

AETHER Lite preserves the strongest ideas of the original AETHER project while reducing implementation complexity. The target simulation is approximately 500 citizens, 150 vehicles, and 50 businesses, with a 2D interface, lightweight concurrency, one active What-If branch, and an optional lightweight prediction module.

Table of Contents

Part I: Foundations & Requirements

1. Foreword	2
2. Executive Summary	2
3. General Introduction	3
3.1 Presentation	3
3.2 Motivation	3
4. Context & Problem Statement	3
4.1 Central Problem	3
4.2 Sub-Problems	3
5. Project Objectives	3
5.1 General Objective	3
5.2 Specific Objectives	3
6. Requirements Analysis	4
6.1 Stakeholders & Actors	4
6.2 Functional Requirements (FR1 to FR10)	4–5
6.3 Non-Functional Requirements	5

Part II: Architecture & Execution

7. Technical Architecture	6
7.1 Architectural Overview	6
7.2 Logical Layers	6
8. Module Descriptions (MOD-01 to MOD-08)	6
9. Technologies & Tools	7
10. Constraints & System Specifications	7
11. Project Planning (Phases 1 to 7)	7
12. Testing & Validation Strategy	8
13. Project Deliverables	8
14. Risk Management	8
15. Conclusion	9
16. Bibliography & References	9
Appendix A — Glossary	9
Appendix B — Conceptual UML & Sign-Off	9

1. Foreword

This document represents the formal Software Requirements Specification (SRS) for AETHER Lite, developed by Louay Hamdi and Yassine Mezghani. The project focuses on Java development, multi-agent simulation, artificial intelligence, data processing, and interactive visualization. The specification defines the objectives, requirements, architecture, technologies, constraints, validation strategy, and expected deliverables.

2. Executive Summary

AETHER Lite is an interactive urban simulation platform that creates a manageable virtual city populated by autonomous citizens, vehicles, and businesses. The system simulates interactions between these entities, records

important events, and allows users to observe the city in real time. A lightweight What-If engine lets the user create one alternative scenario and compare its results with the baseline.

3. General Introduction

3.1 Presentation

AETHER Lite provides a living virtual representation of an imaginary city. It combines a Java simulation engine, autonomous agents, graph-based movement, simplified traffic and economic models, persistent event data, and an interactive 2D visualization.

3.2 Motivation

Cities are complex systems in which transportation, population, businesses, and infrastructure influence one another. AETHER Lite provides a safe environment for demonstrating these interactions while remaining achievable within a student project. The project demonstrates how Java, algorithms, artificial intelligence, databases, and visualization can be combined in one application.

4. Context & Problem Statement

4.1 Central Problem

How can a Java-based application simulate a small but dynamic city in which autonomous entities interact, while allowing users to observe the system and evaluate the consequences of a selected decision?

4.2 Sub-Problems

• Simulating hundreds of autonomous agents without excessive performance degradation.

• Modeling understandable interactions between citizens, vehicles, businesses, and roads.

• Managing simulation events and a useful history of the city.

• Providing graph-based navigation and simplified traffic behavior.

• Creating a reproducible alternative scenario from a saved simulation state.

• Visualizing the simulation in real time without requiring a complex 3D engine.

• Using statistics and an optional lightweight prediction model to identify trends.

5. Project Objectives

5.1 General Objective

To design and implement a Java-based multi-agent urban simulation capable of modeling a small dynamic city, processing simulation events, visualizing its evolution, and evaluating a selected alternative scenario.

5.2 Specific Objectives

• Implement autonomous citizen, vehicle, and business agents.

• Create a configurable city containing roads, neighborhoods, buildings, and services.

• Implement graph-based navigation using A* pathfinding.

• Run a lightweight concurrent simulation of hundreds of agents.

• Generate and process important simulation events.

• Store selected historical states and events.

• Implement one active What-If scenario from a saved state.

• Provide real-time dashboards and a 2D city visualization.

• Calculate KPIs and optionally provide a lightweight prediction feature.

6. Requirements Analysis

6.1 Stakeholders & Actors

Administrator: Configures the city, simulation parameters, population size, and simulation speed.

Analyst: Runs scenarios, compares indicators, analyzes historical data, and studies results.

Observer: Explores the city, views live events, and inspects entities without changing core configuration.

6.2 Functional Requirements (FR1 to FR5)

FR1 — City Configuration	The system shall allow users to create or load a virtual city containing neighborhoods, roads, buildings, and services.
FR2 — Multi-Agent Simulation	The system shall simulate autonomous citizens, vehicles, and businesses whose behavior depends on their state and environment.
FR3 — Citizen Behavior	Citizens shall perform simplified activities such as commuting, working, shopping, and moving between locations.
FR4 — Vehicle & Traffic Simulation	Vehicles shall navigate the road network, select routes, and contribute to simplified congestion indicators.
FR5 — Business & Economy Simulation	Businesses shall manage employees, revenue, expenses, and demand using simplified economic rules.

6.2 Functional Requirements (FR6 to FR10 — Continued)

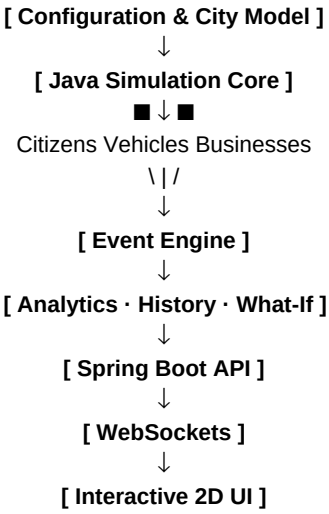
FR6 — Event Processing	The system shall generate and store important events such as movements, transactions, job changes, and infrastructure events.
FR7 — Real-Time Visualization	The system shall display the evolving city, major entities, indicators, and live events while the simulation is running.
FR8 — Simulation Controls	Users shall be able to start, pause, resume, reset, and accelerate the simulation using configurable speed levels.
FR9 — What-If Scenario	Users shall be able to create one alternative scenario from a selected state and introduce a change such as a road closure, business closure, or transport improvement.
FR10 — Scenario Comparison & Analytics	The system shall compare the baseline and alternative scenario using indicators such as traffic, employment, population activity, and economic activity.

6.3 Non-Functional Requirements

NFR-01 Performance	Target a practical configuration of approximately 500 citizens, 150 vehicles, and 50 businesses on development hardware.
NFR-02 Concurrency	Use thread-safe mechanisms for concurrent simulation updates.
NFR-03 Responsiveness	Keep normal dashboard interactions responsive while the simulation runs.
NFR-04 Reproducibility	Support repeatable runs when the same configuration and random seed are used.
NFR-05 Reliability	An individual agent failure should not terminate the complete simulation.
NFR-06 Maintainability	Follow modular design, SOLID principles, and separation of responsibilities.
NFR-07 Usability	Keep controls, indicators, and scenario tools accessible from a unified interface.

7. Technical Architecture

7.1 Architectural Overview



7.2 Logical Layers

Domain & Simulation Layer: agents, city entities, rules, simulation clock, and state transitions.

Algorithm Layer: graph representation, A* pathfinding, traffic calculations, and basic decision logic.

Event & Data Layer: event generation, persistence, history, and KPI calculations.

Decision Layer: What-If scenario evaluation and optional lightweight prediction.

Application Layer: Spring Boot REST services and WebSocket communication.

Presentation Layer: interactive dashboard and 2D city visualization.

8. Module Descriptions

MOD-01 CityManager	Creates and manages city topology, neighborhoods, buildings, roads, and services.
MOD-02 AgentEngine	Creates and updates autonomous citizen, vehicle, and business agents.
MOD-03 RoutingEngine	Maintains the road graph and calculates routes using A* pathfinding.
MOD-04 TrafficEngine	Calculates road usage, congestion, and travel-time indicators.
MOD-05 EconomyEngine	Models business activity, employment, income, and demand using simplified rules.
MOD-06 EventEngine	Generates, distributes, filters, and stores important simulation events.
MOD-07 WhatIfEngine	Creates a state snapshot, applies one scenario change, runs the branch, and compares results.
MOD-08 AnalyticsUI	Calculates KPIs and displays the city, live events, charts, entity details, and controls.

9. Technologies & Tools

Core language	Java 21 LTS
Backend	Spring Boot 3.x
Build & dependencies	Apache Maven
Database	PostgreSQL
Real-time communication	WebSockets
Frontend	React + TypeScript
Visualization	2D web visualization using a suitable chart/canvas/map approach
AI / ML	A Java-compatible lightweight model or simple statistical prediction, if implemented
Testing	JUnit 5 + Mockito
Version control	Git + GitHub
Deployment	Docker / Docker Compose, optional

10. Constraints & System Specifications

10.1 Technical Constraints: The simulation core shall be implemented primarily in Java. The visualization layer should remain replaceable without rewriting the simulation engine. External services are not mandatory for the core simulation.

10.2 Data Constraints: The initial city and events may be generated synthetically. Real-world city data is optional.

10.3 Quality Constraints: The system should support configurable random seeds, logging, validation of agent states, and graceful handling of non-critical failures.

10.4 Hardware Constraints: A standard modern development workstation with approximately 16 GB RAM is recommended. Actual capacity will be benchmarked rather than assumed.

11. Project Planning

11.1 Methodology: Agile/Scrum-style iterations with frequent demonstrations. The working simulation core has priority over optional advanced features.

Phase 1	Analysis & Architecture	Requirements, UML, domain model
Phase 2	City & Agent Core	City model, agents, simulation clock
Phase 3	Algorithms & Events	A* routing, traffic, events, lightweight concurrency
Phase 4	Persistence & Analytics	PostgreSQL, history, KPIs
Phase 5	What-If Engine	Snapshot, one scenario branch, comparison
Phase 6	Visualization	Dashboard, 2D city view, WebSocket updates
Phase 7	AI Extension & Finalization	Optional prediction, tests, documentation, presentation

Advanced 3D visualization, large-scale machine learning, distributed messaging, and external sensor integration are explicitly outside the core AETHER Lite scope.

12. Testing & Validation Strategy

12.1 Test Typology

• Unit tests for agents, routing, economic rules, event processing, and scenario calculations.

• Integration tests for Spring Boot services and PostgreSQL persistence.

• Concurrency tests for simulation updates and shared state.

• Performance benchmarks measuring agents processed per second and memory usage.

• End-to-end tests covering a complete simulation scenario.

12.2 Acceptance Criteria

• The simulation can run a configured city without critical errors.

• Hundreds of agents can be simulated concurrently on the development machine.

• A* routing produces valid paths through the city graph.

• A What-If scenario can be created from a previous state and compared with the baseline.

• Live simulation information is visible in the interface.

• Important historical events can be inspected.

• Core modules have automated tests covering their principal behavior.

13. Project Deliverables

• Complete documented source code.

• Software Requirements Specification (this document).

• UML diagrams and architecture documentation.

• Database schema and initialization scripts.

• Executable application / deployment instructions.

• User documentation.

• Test report and performance benchmark results.

• Final presentation and live demonstration.

14. Risk Management

Simulation becomes too complex	High	Build the core agent engine first and keep advanced features optional.
Performance degradation	Medium	Use profiling, efficient data structures, controlled concurrency, and limited agent counts.
Scenario branching consumes too much memory	Medium	Keep one active branch and store only the required state.
Visualization slows down	Medium	Use aggregation, filtering, and selective rendering.
AI feature takes too long	Low	Keep prediction optional and implement it only after the deterministic simulation works.
Database becomes a bottleneck	Medium	Batch event persistence and separate live state from historical storage.

15. Conclusion

AETHER Lite is designed as a serious Java engineering project rather than a conventional CRUD application. Its central idea is a manageable living city in which individual agent decisions create visible system-level behavior. By combining Java object-oriented design, graph algorithms, event processing, persistence, interactive visualization, scenario branching, and optional AI analytics, the project demonstrates a broad range of software engineering and computer science concepts while remaining achievable as a student project.

The What-If feature is the strongest demonstration element: the user can save a city state, introduce a meaningful change, run an alternative future, and compare its results with the baseline. Historical events and live KPIs reinforce the idea that the city is evolving rather than being a static CRUD dataset.

16. Bibliography & References

• Ferber, J. (1999). Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence. Addison-Wesley.

• Batty, M. (2018). Inventing Future Cities. MIT Press.

• Goetz, B., et al. (2006). Java Concurrency in Practice. Addison-Wesley Professional.

• Russell, S. & Norvig, P. Artificial Intelligence: A Modern Approach.

• ISO/IEC/IEEE 29148. Systems and software engineering — Requirements engineering.

Appendix A — Glossary

Agent: An autonomous software entity that has a state, observes its environment, and performs actions according to defined rules.

Multi-Agent System: A system composed of multiple interacting autonomous agents.

What-If Simulation: A method for creating an alternative future by changing selected variables from a known simulation state.

Simulation Event: A timestamped occurrence that changes or describes the state of the simulated city.

Emergent Behavior: System-level behavior resulting from interactions between individual agents rather than being directly scripted.

Simulation State: The values describing the city and its agents at a particular simulation time.

Appendix B — Conceptual UML

<<interface>> Inspectable

↓

AbstractAgent

■ ↓ ■

CitizenAgent VehicleAgent BusinessAgent

• Citizen: income, occupation, happiness, commute()
• Vehicle: currentSpeed, route, reroute()
• Business: revenue, employees, demand, hire()

The conceptual model can be extended with City, Road, Event, SimulationEngine, Scenario, Snapshot, and Analytics components during implementation.

PROJECT SIGN-OFF

Project: AETHER Lite — Autonomous Multi-Agent Urban Simulation & Decision Intelligence Platform
Creators: Louay Hamdi & Yassine Mezghani · Version 1.0 (Official Baseline Specification)